

PHYS 339 Final Project: The Observation of Brownian Motion on a Macroscopic Scale

Cameron Flaman, Angus Greenwood, Leo Meuller

September 24, 2026

Abstract

Brownian motion is a quintessential example of a stochastic process, which often serves as a foundational introduction to the subject of statistical physics for students. This project aims to create a physical setup that allows for the observation of Brownian motion on a macroscopic scale. The setup applied an actuating, square frame powered by a DC motor; collisions with the frame imparted momentum to a collection of steel bearing balls creating visible stochastic motion of a larger 3D printed puck. The motion of the puck was tracked and statistical tests were performed, comparing the observed motion of the puck to the theoretical probability distributions of the Brownian motion model. While the setup serves as a strong demonstration of a stochastic process and qualitatively exhibits behavior similar to Brownian motion, its actual motion was found to be much more complex — subject to a combination of super and sub-diffusive processes.

1 Introduction

Brownian motion, first described by botanist Robert Brown in 1827, is the statistically-determined motion of a particle suspended in a fluid due to random collisions with the constituent particles of the solution medium. Brown’s discovery served an important role in confirming the atomic nature of matter in the early 20th century by revealing the random, statistically-characterizable models that govern the microscopic scales of nature. Due to the chaotic and dynamic nature of the magnitude and direction of the medium’s impulse on the particle, predicting the instantaneous motion of the individual particle is nearly impossible, however statistical analysis allows for the prediction of the particles behaviour over more broad time intervals. In the simplest model of Brownian motion, with a relatively large particle suspended in a fluid of smaller molecules, the individual fluid molecules bombard the large particle from random directions at random times resulting in the particle’s random walk through the medium. [1] This lab aims to physically simulate this elementary Brownian motion model on a macroscopic scale and explore to what extent a system consisting of macroscopic particle analogs obeys the microscopic, molecular model of Brownian motion under statistical scrutiny.

1.1 The Experiment

To explore two-dimensional Brownian motion on a macroscopic scale, the following experiment will employ an ensemble of small steel bearing balls acting as an analog to the particles of fluid medium. A larger, rolling puck will serve as the particle of interest; photo detection will be applied to track this puck’s displacement and characterize its motion. To create motion chaotic of the steel balls and encourage collision, the balls will be placed in an actuating frame; four moving walls imparting pseudo-random impulse to the balls. The speed of the actuation and the subsequent increase of average particle velocities and collision rate, is analogous to the temperature of the fluid.

1.2 Theory of Brownian Motion

In 1905 Albert Einstein published a paper on the theory of Brownian motion, giving both the theoretical description of it in terms of the collisions between atoms and molecules, and demonstrating that the probability distribution for the position of n Brownian particles follow the macroscopic diffusion equation:

$$\frac{\partial f}{\partial t} = D \frac{\partial^2 f}{\partial x^2} \quad (1)$$

where f is the number density of the n particles and D is the diffusion coefficient [2]. The solution to this equation is

$$f(x; t) = \frac{n}{\sqrt{4\pi Dt}} e^{-x^2/4Dt} \quad (2)$$

from which one can see that the probability distribution for position of a single particle following Brownian motion in one dimension is obtained from simply setting $n = 1$ in this equation.

$$\mathcal{P}_{1D}(x; t) = \frac{1}{\sqrt{4\pi Dt}} e^{-x^2/4Dt} \quad (3)$$

Then, to obtain the probability distribution for the distance travelled in two dimensions of a large particle (assuming the motion in the x and y directions is independent) one can first multiply the probability distributions in x and y together, convert to polar coordinates, and then integrate over the circumference of a circle of radius r as follows:

$$f_{2D}(x, y; t) = f(x; t)f(y; t) = \frac{1}{4\pi Dt} e^{-(x^2+y^2)/4Dt} \quad (4)$$

$$(x, y) \longrightarrow (r \cos \theta, r \sin \theta) \implies f_{2D}(r, \theta; t) = \frac{1}{4\pi Dt} e^{-r^2/4Dt} \quad (5)$$

$$\implies \mathcal{P}(r; t) = \int_0^{2\pi} f_{2D}(r, \theta; t) r d\theta = \int_0^{2\pi} \frac{1}{4\pi Dt} e^{-r^2/4Dt} r d\theta \quad (6)$$

$$\mathcal{P}_{2D}(r; t) = \frac{r}{2Dt} e^{-r^2/4Dt} \quad (7)$$

, where Equation 7 is the probability density function for the 2D distance r the particle has travelled after time t .

1.3 Anomalous Diffusion

In Brownian motion, one can observe a linear relationship between the mean square displacement $\langle r^2(\tau) \rangle$ as a function of the time τ of the random walk [2]. However, in cases of anomalous (non-Brownian) diffusion this relationship becomes non-linear, often following a power law of the form:

$$\langle r^2(\tau) \rangle = K \tau^\alpha \quad (8)$$

, where K is the generalized diffusion coefficient and α is the parameter that governs the type of anomalous diffusion observed. If $\alpha = 1$, simple Brownian motion is recovered. In the case $\alpha > 1$ the system is considered to be super-diffusive and can follow a heavy tailed probability distribution where long jumps in a single time step are possible; e.g. active cellular transport can be described using super-diffusive models. Finally, if $\alpha < 1$ the system is said to be sub-diffusive, and the motion of particles is restricted due to walls of the system or crowding of the particles in the system. In this case the random walk can still freely take small steps but large random steps become restricted. [3] [4]

2 Experimental Methods

The experimental setup devised to produce and record the macroscopic Brownian motion consists of an actuating frame, metal bearing-balls, the central puck, and a webcam. An Elegoo Arduino UNO R3 facilitates frame actuation, and a PC was used to control the Arduino and perform image recognition.

2.1 Physical Components

Pictured in Figures 10 and 11, to assure a level collision-surface the constructed apparatus is mounted on a pair of ThorLabs 12x18x2.28 inch optical breadboards atop a large levelling scale. The actuating frame consists of four 3D printed actuator feet, with one designed to house a JGB-37-520 12V DC motor capable of up to 530 rotations per minute. An aluminium plate is mounted upon 3 stilts with adjustable height to allow for its leveling, with a glass pane placed on top to reduce friction and assure a flat collision-plane. Fastened to the actuator feet by 3D printed actuating corners are four sections of aluminum C-channel, lined with rubber bands to create the 8x9 inch rectangular frame. All rotating components (the top section of the feet and frame corners) are equipped with ball bearings to ensure smooth, low-friction motion. Clamped on a stand above the breadboard is a j5 Create USB Webcam, connected to the PC. Finally, the actuation is controlled by an Arduino UNO R3 via an L298n motor driver wired as in Figure 1 and pictured in Figure 12.

To serve as analogs to the fluid particles of Brownian motion, fifty 1.28 cm diameter steel bearing balls are placed atop the glass pane within the actuation frame. The particle of interest consists of a 3D printed cylindrical puck with a 4.4 cm diameter and a 1.18 cm height, containing three protruding bearing balls to allow it to roll upon the glass and a metal bumper to increase collision elasticity (Figure 9).

2.2 Frame Actuation

The Arduino, DC power supply, motor driver, and DC motor were wired as in Figure 1, with the DC motor embedded in one leg of the actuation frame and able to rotate a corner of the frame in a circle about the axis of the motor. A simple Arduino script allowed for the control of the motor speed at a constant power supply voltage by supplying a PWM signal from pins 7 and 5 of the Arduino. Additionally, the signal from the motor encoder provided a DC pulse 330 times per rotation of the motor, which is read using an attached interrupt on pin 2 of the Arduino. These pulses are counted and used by the Arduino to determine the average rotations per minute the motor underwent in the past second, allowing for the confirmation of a stable RPM of the motor.

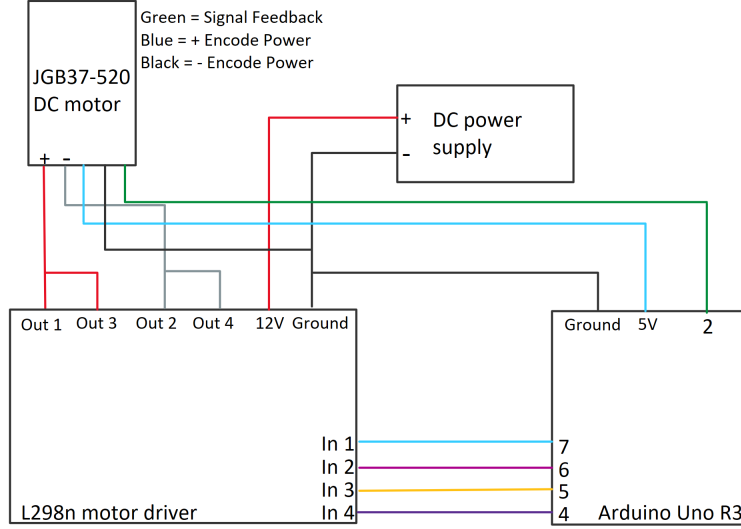


Figure 1: Schematic wiring diagram for the DC motor, Arduino Uno R3, L298n motor driver, and DC power supply. The Arduino receives power from the external PC and the numbers on the Arduino refer to its pins. The legend next to the DC motor corresponds to the labels for the blue, green, and black wires connected to its encoder.

2.3 Live Photo-analysis

Broadly, the photo-detection setup consists of the j5 Create USB Webcam with a USB connection to a computer, which applied numerous OpenCV Python library functions. The camera had a fish-eye lens, which caused image stretching in the extremities of the camera's field of vision; a camera calibration function was applied to eliminate this distortion preventing the inflation of displacement values as the puck approaches the edges of the frame. The calibration code, provided in [Appendix B](#), captures an image of a checkerboard of known-dimension, and flattens the the field of view to restore the even-dimension of the checkerboard tiles in the final image. Additionally, this calibration provided a means of unit conversion from camera pixels to meters, determined to be 2 pixels per millimeter, through the comparison of checkerboard width in pixels and measured width in millimeters.

The top of the puck was painted red to give it a distinct colour compared to the rest of the setup, and allow a simple RGB value mask to locate all pixels that correspond to the puck. To account for motion blur and distortion, blob detection was applied and the centre-value of the puck's color was found by averaging the location of the blob pixels. Data was outputted as a series of x and y values, corresponding to the position of the puck in pixels. During a trial, every time interval Δt the current x and y coordinates were compared to the previous entries to calculate the displacement of the puck over the most recent Δt , with the x, y, and 2D displacements saved to separate lists upon which the data analysis can be performed.

For the live observation of the puck, a circular ring centred on the puck was traced on the live-outputting webcam image feed. The displacement of the centre values was recorded as small blue dots; this allowed the qualitative observation of the data's spread to help assure the puck was not biased to a specific region of the collision surface. This live feed allowed for the observation of initial pooling and leveling issues, explored further in section 2.4, by areas of increased and decreased blue-point density.

2.4 Setup Optimizations

Following initial trials, adjustments were made to the experimental apparatus to improve the quality of measurements. The surface of the experiment consisted of a $\sim 6\text{mm}$ thick glass panel resting atop a metal plate mounted at 3 points — the heights of which could be adjusted independently to allow for the leveling of the plate. A glass surface was chosen due to its uniformity and flatness, minimizing the presence of any low or high points that could not be eliminated through simple leveling of the base. The puck consisted of a 3D-printed shell with a machined aluminium band around its outer edge to make collisions with the bearing balls as elastic as possible, and cavities to hold 3 bearing balls on its bottom allowing the puck to roll to both minimize its friction with the glass pane and attempt to match its friction with the surface to that of the bearing balls.

Two frame sizes were initially tested, of 11x12 inches and 8x9 inches, with the 8x9 inch frame eventually chosen as it was found that the balls tended to clump in the centre of the larger frame — an issue qualitatively eliminated on the smaller frame. Following initial trials, and the breaking of a frame corner, stronger re-enforced frame walls and corners were constructed to assure that the setup can withstand long, high-RPM trials. Additionally, an elastic band was wrapped around the inner walls of the frame to increase the elasticity of the collisions of the bearing balls and the frame.

2.5 Data Acquisition

Running in tandem to the photo-analysis was a dynamically-updating data collection and plotting mechanism. An adjustable delay between displacement calculations was applied to reduce the effect of any frame-rate fluctuations of the camera, and to prevent the live plotting code from being overwhelmed by data from every frame from the camera. Once every second, each pair of coordinates returned by the photo-capture code was used to find the displacements Δx and Δy by taking the difference of the final element of the coordinate list and the element at the previous index. Then, these displacement components were added in quadrature to find the total displacement Δr . The resulting data from the output lists was used for the statistical analysis of the puck's motion.

2.6 Characterizing the Diffusion

The un-normalized Δx and Δy histogram counts were fit to a normal distribution as in Equation 3 and the Δr distribution was fit to Equation 7 with an amplitude parameter A such that

$$\mathcal{P}_{1D}(\Delta x, y; \Delta t) = \frac{A}{\sqrt{4\pi D\Delta t}} \exp\left[\frac{-(\Delta x, y)^2}{4D\Delta t}\right] \quad (9)$$

$$\mathcal{P}_{2D}(\Delta r; \Delta t) = \frac{A\Delta r}{2D\Delta t} \exp\left[\frac{-(\Delta r)^2}{4D\Delta t}\right] \quad (10)$$

where $\Delta x, y$ are the experimentally-acquired x and y displacement values after one time step, $\Delta r = \sqrt{\Delta x^2 + \Delta y^2}$, and Δt is the evenly-spaced one-second sampling period. The assignment of bin number was performed as to ensure that more than 80% of bins contained more than five counts; this is a standard count minimum for a χ^2_{red} test to be performed on histogram count data. [5]

In order to conclude upon the quality of the fit, residuals were calculated according to $R = C_{\text{exp}} - C_{\text{obs}}$ where C_{exp} were the expected counts resulting from the fits to Equations 3 and 7, and C_{obs} were the counts of the experimentally-obtained histogram. A reduced-chi-squared (χ^2_{red}) quality of fit test was then performed according to

$$\chi^2 = \sum_{\alpha} \left(\frac{R_{\alpha}^2}{C_{\text{exp}}} \right) \quad (11)$$

$$\chi^2_{\text{red}} = \frac{\chi^2}{N_{\text{bin}} - N_{\text{param}} - 1} \quad (12)$$

where R_{α} are the respective residual values for each bin α , N_{bin} is the number of bins in the histogram and $N_{\text{param}} = 2$ is the number of fitting parameters.

Along with the fits to the probability distributions for Brownian motion described above, the distance traveled by the puck at $\sim 460\text{RPM}$ after various time steps in the range 100–2000ms was measured, with 250 measurements taken at each time step. This was used to calculate the mean square distance ($\langle r^2(\tau) \rangle$) for the varying time steps (τ), which were then fit to the power law in Equation 8 to determine the α parameter of the system. An α value close to 1 would indicate that the motion of the system was likely well described as Brownian, whereas α values deviating

from 1 would indicate that the system experienced anomalous diffusion. [3] Finally, a Q-Q test was applied to the experimental probability distribution to give a further qualitative assessment of how well the Brownian motion model describes the motion of the puck. All curve fitting was performed using the `scipy.optimize.curve_fit` Python function,

3 Results

The initial results of the experiment, plotted according to the procedure detailed in Section 2.5 returned Figure 2 below for the 2D displacement for the puck. The corresponding X and Y displacement distributions are pictured in Appendix A, fit to their respective one-dimensional probability models as in Equation 3.

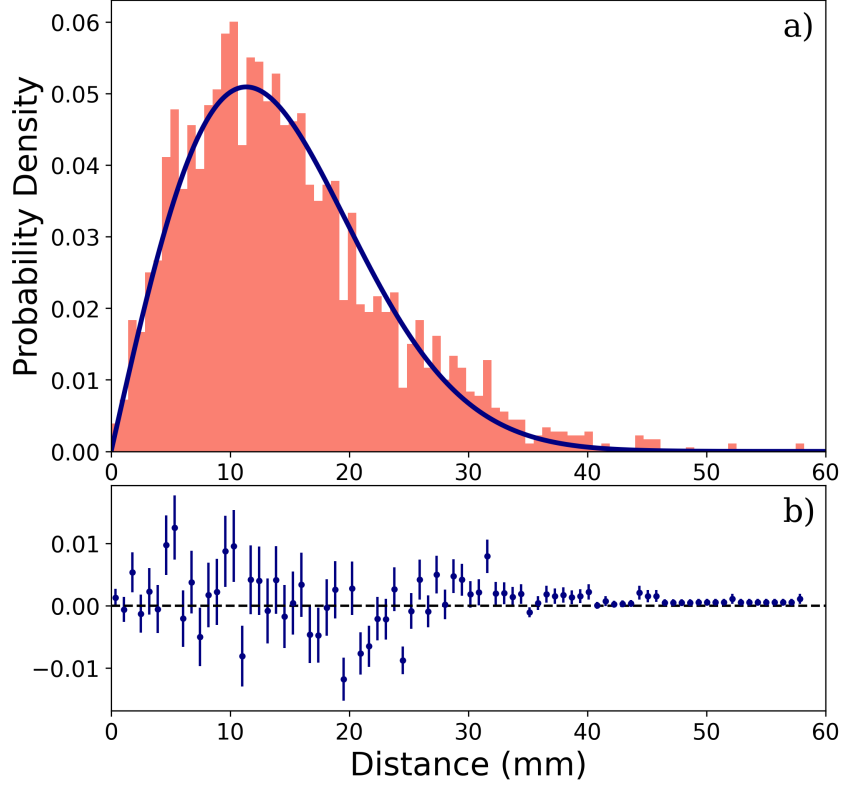


Figure 2: a): Normalized 82-bin histogram of puck distance values, recorded every time interval $\Delta t = 1$ second for a total of $T = 2534s$. Pictured in pink is the probability density of the distance, across the x axis. In purple, the fit of the bin centers to $\mathcal{P}_{2D}$ as described in Equation 9 with parameters $A = 1714 \pm 34$, $D = 64 \pm 1$ before normalization. b): Plotted below are the residuals of the fit to the bin centers, with error bars corresponding to Poissonian counting error (discussed in Section 4.1). The fit returned a $\chi^2_{red} = 55.012$ by applying the test according to Equation 12.

As made evident by the large χ^2_{red} values and patterns in the residuals for all three fits, the theoretical model fails to properly fit the experimentally-obtained data. A Q-Q test was then applied to serve as a more descriptive indication of the failures of the probability distribution model to represent the data set. Applying the Q-Q test with a normal target distribution to the Horizontal component of motion gave a distinct “S” shape about the ideal, linear distribution [6] as pictured in Figure 3. This suggests that the experimental distribution contains a “heavy-tail”; extreme displacement-value counts much greater than the predicted values from the distribution.

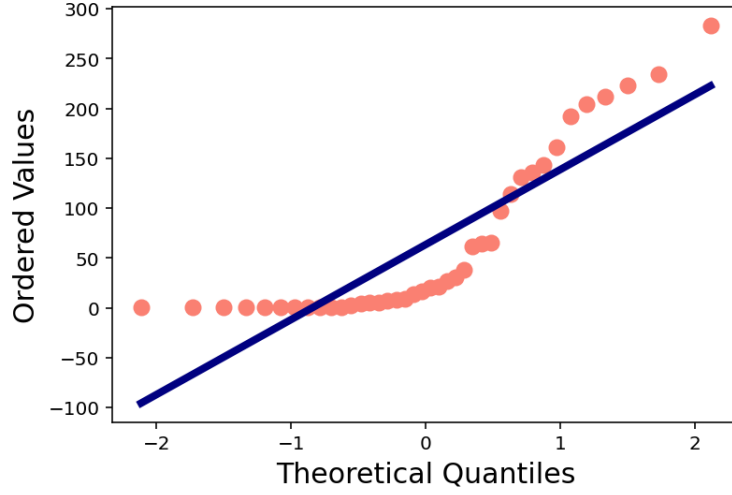


Figure 3: Q-Q plot comparing evenly-spaced quantiles of a theoretical normal distribution on the vertical axis and quantiles (points below which a certain percentage of data is expected to fall [6]) of the experimentally-obtained distribution of the horizontal displacement of the puck on the x-axis.

3.1 Diffusion Behaviour

250 measurements of the distance travelled after each time step τ of the puck at 460 RPM were used to calculate the mean-square-distance travelled by the puck for that τ , with the error on the data point being calculated as the standard error on the mean of the 250 measurements. This was repeated for 19 different τ values in the range 100–2000ms, and plotted in Figure 4 below. A curve fit was performed to the function in Equation 8 to determine the power law followed by diffusion, resulting in parameters of $K = 0.0568 \pm 0.009$ and $\alpha = 0.94 \pm 0.02$, and a reduced chi-squared of $\chi^2_{\text{red}} = 0.040$. These results indicated that the diffusion is likely sub-diffusive, since $\alpha < 1$, and that the fitting function is over-fitting the data; likely due to an overestimation of the error on the data points.

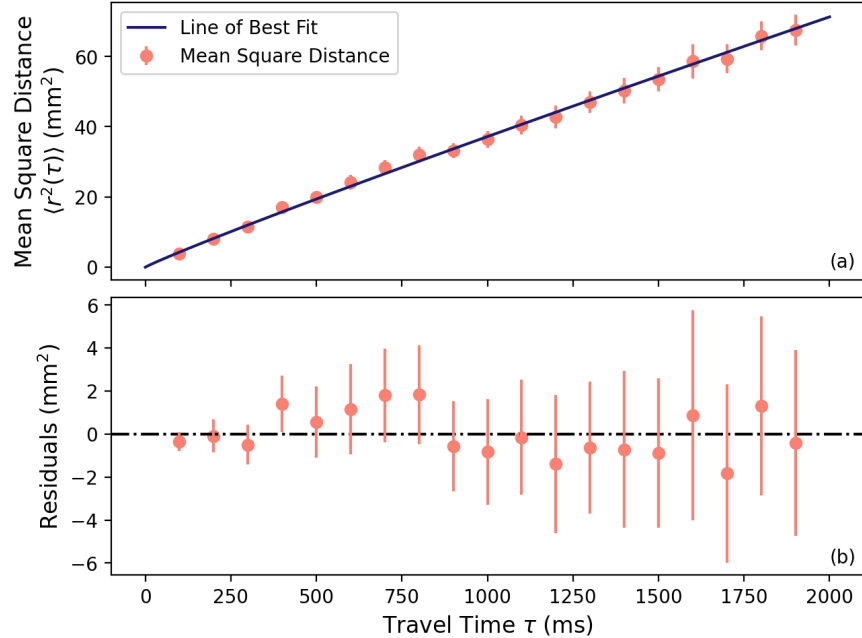


Figure 4: a) Plot of the mean-square-distance travelled by the puck (in 2D) as a function of the time between distance measurements (τ). The best fit function is of the form $\langle r^2(\tau) \rangle = K\tau^\alpha$ and gave parameter values of $K = 0.0568 \pm 0.009$ and $\alpha = 0.94 \pm 0.02$ with a reduced chi-squared value of $\chi^2_{\text{red}} = 0.040$. The data points are means of 250 measurements taken at each time step, and the error bars are the standard error of the mean of said 250 measurements; calculated using the `scipy.stats.sem` function in Python. b) Residuals to the line of best fit.

4 Discussion

4.1 Error Propagation and Data Binning

The displacement measurements are theoretically discrete, independent events, recorded over even time-intervals; the counts of each bin are never negative, and thus it was concluded that their error should conform to Poissonian counting statistics. From this the standard counting error for a bin of the histogram is given by $\sqrt{N}$ where N is the amount of counts in the bin. This error provided a vertical error on the centre points of the bin for the purpose of residual analysis to the theoretical probability curves. [5]

4.2 Heavy Tail of the Experimental Displacement PDF

It is clear that the greatest discrepancy in the experimental results to theoretical values, and subsequently the regime with the most contribution to the large χ^2_{red} of the 2D displacement PDF in Figure 2, lies in the extremities of the displacement values. A much larger quantity of relatively large displacement values were found than would be predicted by the theoretical PDF (Equation 7), lending the experimental distribution a “heavy-tail” uncharacteristic of Brownian motion. To confirm that the tail of the distribution is the area that is most inconsistent with the expected PDF for Brownian motion, the fit was performed to the same data as in Section 3 but with a cutoff applied to the tail of the distribution at 40mm (filtering out all displacement measurements beyond this point). This filtered data was then fit to the same probability model, giving parameters of $A = 1113 \pm 23$, and $D = 63 \pm 1$ prior to normalization and a reduced chi-squared value of 1.652, with no obvious trend to the residuals, as shown in Figure 5 below. Additionally, a second Q-Q plot of the x-displacements was performed for the filtered data, displayed in Figure 6 leading to a curve that is qualitatively much more linear than the one in Figure 3, which is the expected behaviour for a Q-Q plot where the experimental and theoretical models agree. [6]

Altogether these results indicate that neglecting the heavy tail of the raw data leads to a much closer agreement with the expected model for Brownian motion. This suggest certain lapses in the analogy of the system to true Brownian motion; factors neglected by the simple Brownian motion model which may play a role in the setup constructed and cause deviations from the Brownian model are further explored in Section 4.4.

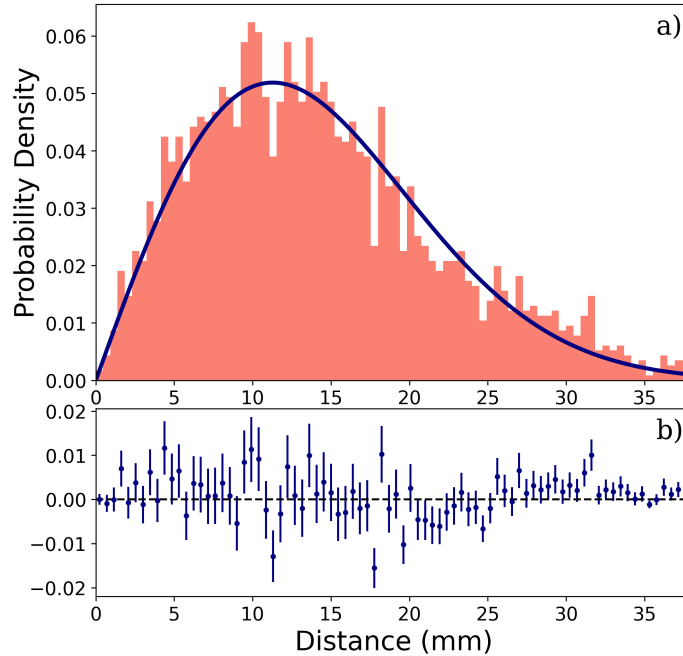


Figure 5: a): Filtered, normalized 82-bin histogram of puck distance values, recorded every time interval $\Delta t = 1$ second for a total of $T = 2534s$. Pictured in pink is the probability density of the distance, across the x axis. In purple, the fit of the bin centers to $\mathcal{P}_{2D}$ as described in Equation 9 with parameters $A = 1113 \pm 23$, $D = 63 \pm 1$ before normalization. b): Plotted below are the residuals of the fit to the bin centers, with error bars corresponding to Poissonian counting error. All counts of displacement values greater than 40mm removed. The fit returned a $\chi^2_{\text{red}} = 1.652$ by applying the test according to Equation 12.

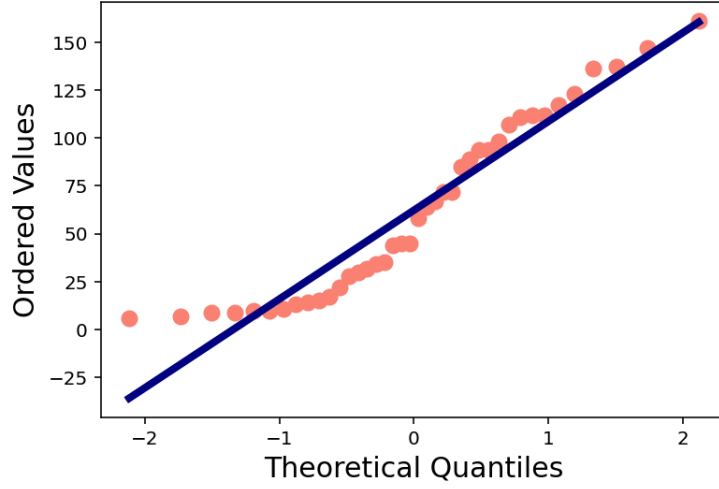


Figure 6: Q-Q plot comparing evenly-spaced quantiles of a theoretical normal distribution on the vertical axis and quantiles of the filtered, experimentally-obtained distribution of the horizontal displacements of the puck on the x-axis.

-	X-Y Distance χ^2_{red}	X Displacement χ^2_{red}	Y Displacement χ^2_{red}
Un-Filtered Data	55.012	16.826	16.496
Filtered Data	1.652	1.197	1.218

Table 1: Tabulated data comparing goodness of fit χ^2_{red} tests for experimental data, performed according to the test described in 12 with respective models described in Equations 7 and 3.

4.3 Diffusion Behaviour

The results of the mean-square-displacement test (Figure 4) indicate that the motion of the puck is likely sub-diffusive due to the power of $\alpha = 0.94 \pm 0.02$. This makes qualitative sense as the limitations imposed by the frame and the balls in the setup on the motion of the puck, further described in section 4.4, likely favours sub-diffusive behaviour [3] [4]. However, this result contradicts the results discussed above in Section 4.2, which indicates that the motion of the puck is likely super-diffusive thanks to the heavy tail of the experimental displacement PDF. This discrepancy is discussed further in the following section (4.4).

4.4 Sources of Error in the Physical Experimental Setup

The assumption of Brownian motion taking place in a large, continuous medium relative to the particle of interest makes it difficult to replicate on the size scale attempted in this experiment. The actuating frame had to be restricted to a relatively small size compared to the puck and steel balls in order to continuously impart momentum and prevent stalling and pooling of the bearing balls as mentioned in Section 2.4. Thus, as the puck drifts towards the edges of the frame, it experiences greater impulse from both collisions with the balls, as well as potential collisions with the frame itself. These edge-cases imparts significantly greater momentum to the puck, resulting in extreme, unexpected displacements. The extreme distance counts recorded due to the proximity to the edge could be contributing to the observed heavy tail of the experimental distribution. Meanwhile, the relatively small size ratio of the frame to the puck compared to e.g. a grain of pollen suspended in water restricts the macroscopic motion of the puck — a condition and behaviour characteristic of some sub-diffusive systems.

Additionally, the assumption of small, point-mass particles making up the medium enables a greater number density of particle concentration than was able to be realistically achieved in the lab. This experimental deviation from the Brownian model likely caused a decrease in the head-on collision rate between the puck and the balls, potentially allowing the puck to slide further un-interrupted along the plane than predicted by the model. This would cause the resultant displacement values to sometimes be larger than expected, potentially contributing to the observed heavy-tail. On the other hand, because Brownian motion assumes that the collision particles are point-particles with infinitesimally small radii relative to the particle of interest, the volume occupied by the steel balls is a non-negligible deviation from this ideal Brownian model. The radii of the balls may cause collisions and drag not present in the idealized Brownian models, thereby reducing the freedom of motion of the puck and contributing to the sub-diffusive behaviour indicated by the mean-square-distance test. It was not possible to determine whether

the non-point-nature of the steel balls was more likely to lead to super or sub-diffusive behaviour, but it certainly represents an approximation made by the simple Brownian model that contributes to the discrepancy between predicted and experimental outcomes.

Finally, the rotational motion that is imparted to both the puck and the balls is a difficult-to-characterize factor of the experimental setup not contained within the idealized model of Brownian motion. This would remove translational energy from the system, damping the motion of the puck and deflating the distance values, potentially contributing to the sub-diffusive nature of the collected data along with the aforementioned volume of the bearing balls. [7]

5 Conclusion

The extreme displacement counts obtained, and the subsequent heavy-tail nature of the experimental distribution seems to contradict the observation of the restricted motion of a sub-diffusive stochastic process in the mean-square-distance test. The nature of the setup has potential to either increase and restrict the freedom of motion and momentum imparted onto puck depending on its position: the proposed processes that restrict particle motion primarily occur when the puck is located in a central-frame position, whereas the large anomalous displacement phenomena likely occur near the edge of the actuation frame. During the stochastic movement of the puck the actuation frame moves the puck with greater impulse as it approaches the walls as it undergoes collisions with balls that have maximal initial momentum from collisions with the frame. This pushes the puck more rapidly to the centre of the frame and results in the long jumps evident in the heavy-tail of the displacement PDF. However, while the puck in the centre, the additional drag from the non-point nature of the steel balls along with the limitations of the frame size may lead to the dominant sub-diffusive behaviour observed in the mean-square-distance plot.

In order to simulate motion more that conforms more to ideal the Brownian motion model and reduce aspects of the setup that cause the aforementioned contradictory outcomes, numerous adjustments to the physical setup could be made. A larger frame would serve to improve upon the consistency of the puck behaviour; the puck would spend much more time in the sub-diffusive regimes, thus large displacements would become less frequent. Additionally, adjusting the relative size of the puck to the steel balls will serve as a better analog to Brownian conditions with a point-particle medium. This modification would likely approach Brownian diffusion by reducing sub-diffusive effects; as the puck will be subject to decreased drag and resistance of a smaller-particle medium. The inclusion of a greater number density of smaller balls would more-accurately replicate the continuous medium assumed by Brownian motion, and could potentially aid the model in satisfying Brownian motion by reducing anomalously-large displacements, while simultaneously providing more impulses to the puck.

Despite the experiment’s failure to replicate ideal Brownian motion under statistical tests, this exploration serves to demonstrate the value of this setup as a macroscopic example of a complex statistical processes. The constructed system serves as a stochastic-sandbox where one can explore the nature of diffusion, and how the modification of various factors such as frame rotation rate, particle size, and particle density can effect the outcome of the experiment. From an educational perspective, the ball-model of Brownian motion serves as a great demonstration of the stochastic processes present in areas such as statistical mechanics and biophysics. Despite the deviance of the displacement distribution from the idealized models, the obtained distributions are qualitatively very similar to what was expected. Applying an animated histogram, one can observe the central limit theorem slowly guide the stochastic process to the underlying statistical models that govern these crucial physics processes. As a physics outreach and pedagogical tool, the setup serves an incredibly engaging and clear demonstration.

References

- [1] M. J. Kozdron, “Brownian motion and the heat equation,” <https://uregina.ca/~kozdron/Research/UgradTalks/BM.and.Heat/heat.and.BM.pdf>, archived: 26 November, 2024. Retrieved: 9 April, 2026. [1](#)
- [2] A. Einstein, “Über die von der molekularkinetischen theorie der wärme geforderte bewegung von in ruhenden flüssigkeiten suspendierten teilchen,” *Annalen der physik*, vol. 4, 1905. [2](#)
- [3] G. Muñoz-Gil, G. Volpe, M. A. Garcia-March, E. Aghion, A. Argun, C. B. Hong, T. Bland, S. Bo, J. A. Conejero, N. Firbas *et al.*, “Objective comparison of methods to decode anomalous diffusion,” *Nature communications*, vol. 12, no. 1, p. 6253, 2021. [2](#), [5](#), [8](#)

- [4] B. M. Regner, D. Vučinić, C. Domnisoru, T. M. Bartol, M. W. Hetzer, D. M. Tartakovsky, and T. J. Sejnowski, “Anomalous diffusion of single particles in cytoplasm,” *Biophysical journal*, vol. 104, no. 8, pp. 1652–1660, 2013. 2, 8
- [5] C. Fitzgerald, “Chi squared test,” <https://webpages.csus.edu/fitzgerald/chi-squared-test/>, updated: 1 March, 2021. Retrieved: 23 April, 2026. 4, 7
- [6] C. Ford, “Understanding qq plots,” <https://library.virginia.edu/data/articles/understanding-q-q-plots>, archived: August 26, 2015. Retrieved: 23 April, 2026. 5, 6, 7
- [7] U. Basu, S. N. Majumdar, A. Rosso, and G. Schehr, “Active brownian motion in two dimensions,” *Physical Review E*, vol. 98, no. 6, p. 062121, 2018. 9

Appendix A Supplementary Figures

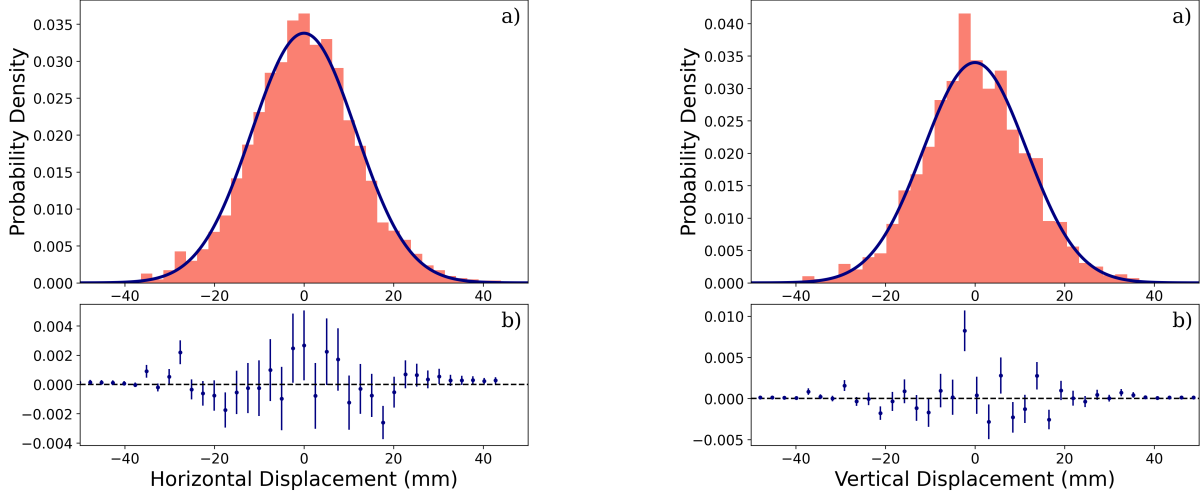


Figure 7: a): Normalized 40-bin histograms of puck distance values, for the horizontal (left) and vertical components (right) recorded every time interval $\Delta t = 1$ second for a total of $T = 2534s$. Pictured in purple is the probability density of the distance, across the x axis. In pink, the fit of the bin centers to $\mathcal{P}_{1D}$ as described in 9 with parameters $A = 6281 \pm 126$, $D = 11.7 \pm 0.2$ for the horizontal displacements, and $A = 6673 \pm 134$, $D = 11.5/pm0.2$ for the vertical. b): Plotted below are the residuals of the fit to the bin centers, with error bars corresponding to Poissonian counting error. The fit returned $\chi^2_{red} = 16.826$ and $\chi^2_{red} = 16.496$ for the left and right plots respectively.

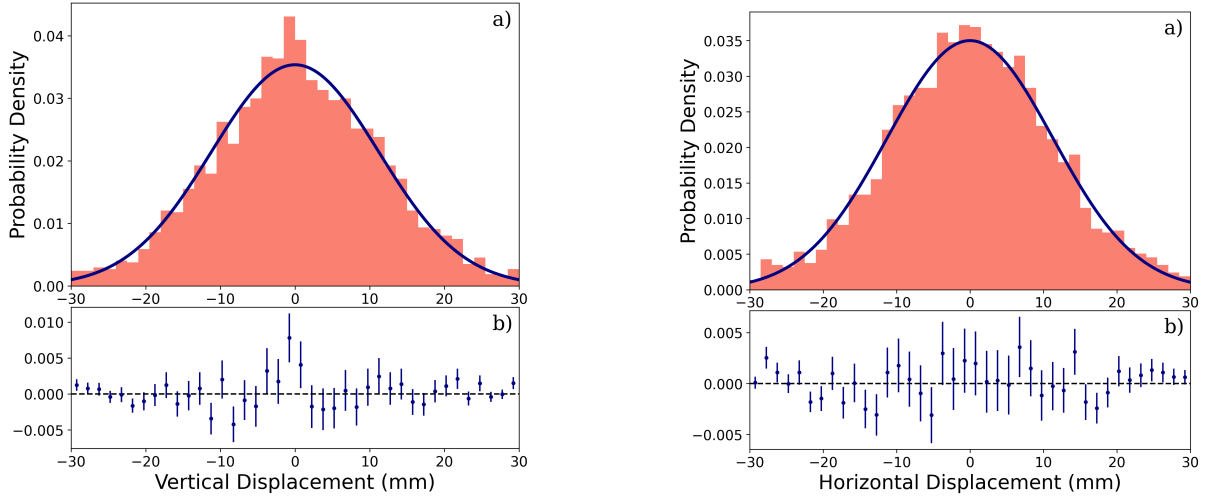


Figure 8: a): Filtered, normalized histograms of puck distance values, for the horizontal (left) and vertical components (right) recorded every time interval $\Delta t = 1$ second for a total of $T = ???s$. Pictured in purple is the probability density of the distance, across the x axis. In pink, the fit of the bin centers to $\mathcal{P}_{1D}$ as described in 9 with parameters $A = 3716 \pm 75$, $B = 11.3 \pm 0.2$ for the horizontal displacements, and $A = 3710 \pm 75$, $B = 11.2 \pm 0.2$ for the vertical. b): Plotted below are the residuals of the fit to the bin centers, with error bars corresponding to Poissonian counting error. The fit returned $\chi^2_{red} = 1.197$ and $\chi^2_{red} = 1.218$ for the left and right plots respectively.

Appendix B Code

```

# -*- coding: utf-8 -*-
"""
Created on Mon Mar 30 15:03:24 2026

@author: lmuell
"""
import numpy as np
import cv2 as cv

cap = cv.VideoCapture(0, cv.CAP_DSHOW)
offset = 10
centers = []

cap.set(cv.CAP_PROP_EXPOSURE, -4)

ret, frame = cap.read()
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....., (6,5,0)
objp = np.zeros((12*12,3), np.float32)
objp[:, :2] = np.mgrid[0:12, 0:12].T.reshape(-1,2)

# Arrays to store object points and image points from all the images.
objpoints = [] # 3d point in real world space
imgpoints = [] # 2d points in image plane.

gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
lowerLimit, upperLimit = np.array([0,0,70]), np.array([180,255,255])

ret, corners = cv.findChessboardCorners(gray, (12,12), None)

# If found, add object points, image points (after refining them)
if ret == True:
    objpoints.append(objp)

    corners2 = cv.cornerSubPix(gray, corners, (11,11), (-1,-1), criteria)
    imgpoints.append(corners2)

    # Draw and display the corners

    #cv.imshow("fart", frame)
else:
    print('fart')

ret, mtx, dist, rvecs, tvecs = cv.calibrateCamera(objpoints, imgpoints, gray.shape
h, w = frame.shape[:2]
newcameramtx, roi = cv.getOptimalNewCameraMatrix(mtx, dist, (w,h), 1, (w,h))
frame = cv.undistort(frame, mtx, dist, None, newcameramtx)

```

```

x, y, w, h = roi

frame = frame[y:y+h, x:x+w]
#cv.imshow("fart",gray)
cv.imshow("dewarp",frame)
print("good")

gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)

ret, corners = cv.findChessboardCorners(gray, (12,12), None)

pts1 = np.float32([[int(corners[0][0][0]),int(corners[0][0][1])],
                  [int(corners[11][0][0]),int(corners[11][0][1])],
                  [int(corners[132][0][0]),int(corners[132][0][1])],
                  [int(corners[143][0][0]),int(corners[143][0][1])]])

pts2 = np.float32([[offset,offset],[400-offset,offset],[offset,400-offset],[400-o

M = cv.getPerspectiveTransform(pts1,pts2)

dst = cv.warpPerspective(gray,M,(400,400))

cv.imshow("perspective",dst)

cv.waitKey(5000)
cap.release()

cv.destroyAllWindows()

```

```

# -*- coding: utf-8 -*-
"""
Created on Tue Apr  7 15:00:21 2026

@author: lmuell
"""
import cv2 as cv
import numpy as np
from PIL import Image
import time as t
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import os
import csv

os.chdir(r"C:\Users\lmuell\Desktop\LABDATA")
time_interval = 0.5
bin_num = 40
filename = "500ms_50balls_400RPM.csv"
boardmm = 228
#%%
def detect_circles(image):
    _, labels, _, _ = cv.connectedComponentsWithStats(image)

    # Extract the pixels associated with the current label
    label_mask = np.zeros(image.shape, dtype=np.uint8)
    label_mask[labels == 1] = 255

    params = cv.SimpleBlobDetector_Params()

    # Set Area filtering parameters
    params.filterByArea = True
    params.minArea = 1000
    params.maxArea = 100000

    params.filterByCircularity = True
    params.minCircularity = .7

    # Create a detector with the parameters
    detector = cv.SimpleBlobDetector_create(params)

    # Detect blobs
    keypoints = detector.detect(label_mask)
    circles = []
    for kp in keypoints:
        circles.append(np.array([int(kp.pt[0]), int(kp.pt[1]), int(kp.size / 2)]))

    return circles
#%%

```

```

#%%

offset =30
size = 2*boardmm+60
centers = []

frame = cv.imread('calibrationimage.jpg')

criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....., (6,5,0)
objp = np.zeros((11*11,3), np.float32)
objp[:, :2] = np.mgrid[0:11,0:11].T.reshape(-1,2)

# Arrays to store object points and image points from all the images.
objpoints = [] # 3d point in real world space
imgpoints = [] # 2d points in image plane.

gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
lowerLimit, upperLimit = np.array([170,200,20]), np.array([179,255,255])

ret, corners = cv.findChessboardCorners(gray, (11,11), None)

# If found, add object points, image points (after refining them)
if ret == True:
    objpoints.append(objp)

    corners2 = cv.cornerSubPix(gray,corners, (11,11), (-1,-1), criteria)
    imgpoints.append(corners2)

    # Draw and display the corners

    #cv.imshow("fart",frame)
else:
    print('fart')

ret, mtx, dist, rvecs, tvecs = cv.calibrateCamera(objpoints, imgpoints, gray.shape

h, w = frame.shape[:2]
newcameramtx, roi = cv.getOptimalNewCameraMatrix(mtx, dist, (w,h), 1, (w,h))
frame = cv.undistort(frame, mtx, dist, None, newcameramtx)
x, y, w, h = roi
frame = frame[y:y+h, x:x+w]

#cv.imshow("fart",gray)
#cv.imshow("dewarp",frame)
print("good")

```

```

gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)

ret, corners = cv.findChessboardCorners(gray, (11,11), None)

corners2 = cv.cornerSubPix(gray,corners, (11,11), (-1,-1), criteria)

pts1 = np.float32([[int(corners2[0][0][0]),int(corners2[0][0][1])],
                  [int(corners2[10][0][0]),int(corners2[10][0][1])],
                  [int(corners2[110][0][0]),int(corners2[110][0][1])],
                  [int(corners2[120][0][0]),int(corners2[120][0][1])]])

pts2 = np.float32([[offset,offset],[size-offset,offset],[offset,size-offset],[size-offset,size-offset]])

M = cv.getPerspectiveTransform(pts1,pts2)

dst = cv.warpPerspective(gray,M,(size,size))
print(size-2*offset)

#cv.imshow("perspective",dst)

#%%
cap = cv.VideoCapture(0, cv.CAP_DSHOW)

cap.set(cv.CAP_PROP_EXPOSURE,-4)
time_ints = [100,200,300,400,500,600,700,800,900,1000,1100,1200,1300,1400,1500,1600]
for j in time_ints:
    print(j/2000)
    centers = []
    time_i = t.time()
    disps = []
    x_disps = []
    y_disps = []
    ind_i = 0
    while len(disps)<250:

        ret, frame = cap.read()
        frame = cv.undistort(frame, mtx, dist, None, newcameramtx)
        x, y, w, h = roi
        frame = frame[y:y+h, x:x+w]

        frame = cv.warpPerspective(frame,M,(size,size))

        dst = cv.warpPerspective(gray,M,(size,size))

        hsvImage = cv.cvtColor(frame, cv.COLOR_BGR2HSV)

        lowerLimit, upperLimit = np.array([170,100,100]), np.array([179,255,255])

```

```

mask = cv.inRange(hsvImage, lowerLimit, upperLimit)

mask_ = Image.fromarray(mask)

blur = cv.bitwise_not(cv.medianBlur(mask, 5))

circles = detect_circles(blur)
if circles is not None:
    circles = np.uint16(np.around(circles))
    for i in circles:
        x = i[0]
        y = i[1]
        centers.append([x, y])
        center = (x, y)

        # circle center
        cv.circle(frame, center, 1, (0, 100, 100), 3)
        # circle outline
        radius = i[2]
        cv.circle(frame, center, radius, (255, 0, 255), 3)
    for i in centers:
        cv.circle(frame, (i[0], i[1]), 1, (255, 0, 0), 3)
    #print(circles)

time_f = t.time()
if time_f > time_i + j/1000 and ind_i < len(centers):

    current_pos = np.array(centers[-1], dtype=float)/2
    start_pos = np.array(centers[ind_i], dtype=float)/2

    diff = current_pos - start_pos
    disp = np.sqrt(np.sum(diff**2))
    disps.append(disp)

    x_disp = (current_pos[0] - start_pos[0])
    x_disps.append(x_disp)

    y_disp = (current_pos[1] - start_pos[1])
    y_disps.append(y_disp)

    counts, bin_edges = np.histogram(disps, bins=20)
    bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

    x_counts, x_bin_edges = np.histogram(x_disps, bins=20)
    x_bin_centers = (x_bin_edges[:-1] + x_bin_edges[1:]) / 2

    y_counts, y_bin_edges = np.histogram(y_disps, bins=20)
    y_bin_centers = (y_bin_edges[:-1] + y_bin_edges[1:]) / 2

```

```

        if len(disps)%25 == 0:
            print(len(disps))
            time_i = time_f
            ind_i = len(centers) - 1

    cv.imshow('blur', blur)
    cv.imshow('final image', frame)
    cv.imshow('mask', mask)
    check = cv.waitKey(1)

    if check & 0xFF == ord('c'):
        centers = []

    if check & 0xFF == ord('q'):
        break
    check = False

    print(np.mean(disps)**2)
    np.savetxt(str(j)+"ms_50balls_460RPM.csv", [disps,x_disps,y_disps], delimiter
cap.release()

cv.destroyAllWindows()
# %%

```

```

1  const int IN1 = 7;
2  const int IN2 = 8;
3  const int IN3 = 9;
4  const int IN4 = 10;
5  const int ENABLEA = 6;
6  const int ENABLEB = 11;
7  const int PHASEA = 2;
8  const int PHASEB = 3;
9
10 const int PPR = 330;
11 volatile int count = 0;
12 unsigned long prevTime = 0;
13 float RPM = 0;
14 int check = 0;
15 int speed = 100;
16 void setup() {
17     Serial.begin(115200);
18     attachInterrupt(digitalPinToInterrupt(PHASEA), RPMcounter, RISING);
19     pinMode(IN1, OUTPUT);
20     pinMode(IN2, OUTPUT);
21     pinMode(IN3, OUTPUT);
22     pinMode(IN4, OUTPUT);
23     pinMode(ENABLEA, OUTPUT);
24     pinMode(ENABLEB, OUTPUT);
25     pinMode(PHASEA, INPUT);
26     pinMode(PHASEB, INPUT);
27
28 }
29
30 void loop() {
31     motorControl(0, 1, 300);
32
33     // getRPM(1);
34     // input is number of milliseconds to avg RPM over
35     Serial.println(getRPM(1000, 1));
36 }
37
38 void motorControl(int state, int direction, int selectRPM) {
39     if (getRPM(100,0) < selectRPM && millis() > 100 && speed < 255) {
40         speed+=1;
41         delay(50);
42     } else if (getRPM(100,0) > selectRPM && millis() > 100 && speed > 0) {
43         speed-=1;
44         delay(50);
45     }
46
47     if (state == 1) {
48         if (direction == 1) {
49             digitalWrite(IN1, HIGH);
50             digitalWrite(IN2, LOW);

```

```

51     analogWrite(ENABLEA, speed);
52     digitalWrite(IN3, HIGH);
53     digitalWrite(IN4, LOW);
54     analogWrite(ENABLEB, speed);
55 }
56
57 if (direction == 0) {
58     digitalWrite(IN1, LOW);
59     digitalWrite(IN2, HIGH);
60     analogWrite(ENABLEA, speed);
61     digitalWrite(IN3, LOW);
62     digitalWrite(IN4, HIGH);
63     analogWrite(ENABLEB, speed);
64 }
65 }
66
67 if (state == 0) {
68     digitalWrite(IN1, LOW);
69     digitalWrite(IN2, LOW);
70     analogWrite(ENABLEA, 0);
71     digitalWrite(IN3, LOW);
72     digitalWrite(IN4, LOW);
73     analogWrite(ENABLEB, 0);
74 }
75 }
76
77 float getRPM(int rpmTime, int output) {
78
79     if (millis()%rpmTime==1) {
80         if (check == 0) {
81             noInterrupts();
82
83             RPM = (count * 60000 / PPR) / rpmTime;
84             count = 0;
85             prevTime = millis();
86
87             interrupts();
88             check == 1;
89         }
90     } else {
91         check == 0;
92     }
93
94     return RPM;
95 }
96
97 void RPMcounter() {
98     count++;
99 }
100

```

```
101 // void getRPM(int timestep) {
102 //   count+=(1-digitalRead(PHASEA));
103
104 //   if (millis()%1000==0) {
105 //     if (check == 0) {
106 //       Serial.println(count);
107 //       int revolutions = (count/11);
108 //       RPM = revolutions/60;
109 //       count = 0;
110 //       check = 1;
111 //     }
112 //   } else {
113 //     check = 0;
114 //   }
115
116 // }
117
118
119
```

Appendix C Images of Experimental Setup



Figure 9: Top and bottom view of the 3D printed tracking puck, enclosing three steel bearing balls, and re-enforced by an steel aluminum ring.

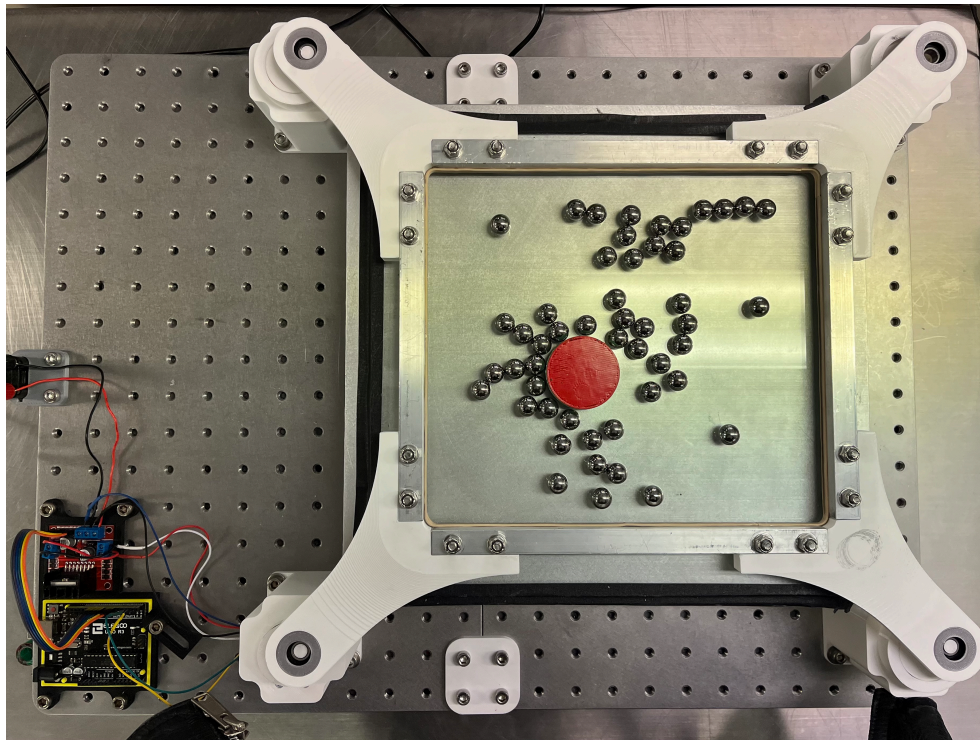


Figure 10: Top-down view of the actuation frame, actuation feet, actuation corners, optical and levelling tables, puck and steel balls, glass surface and Arduino connection.

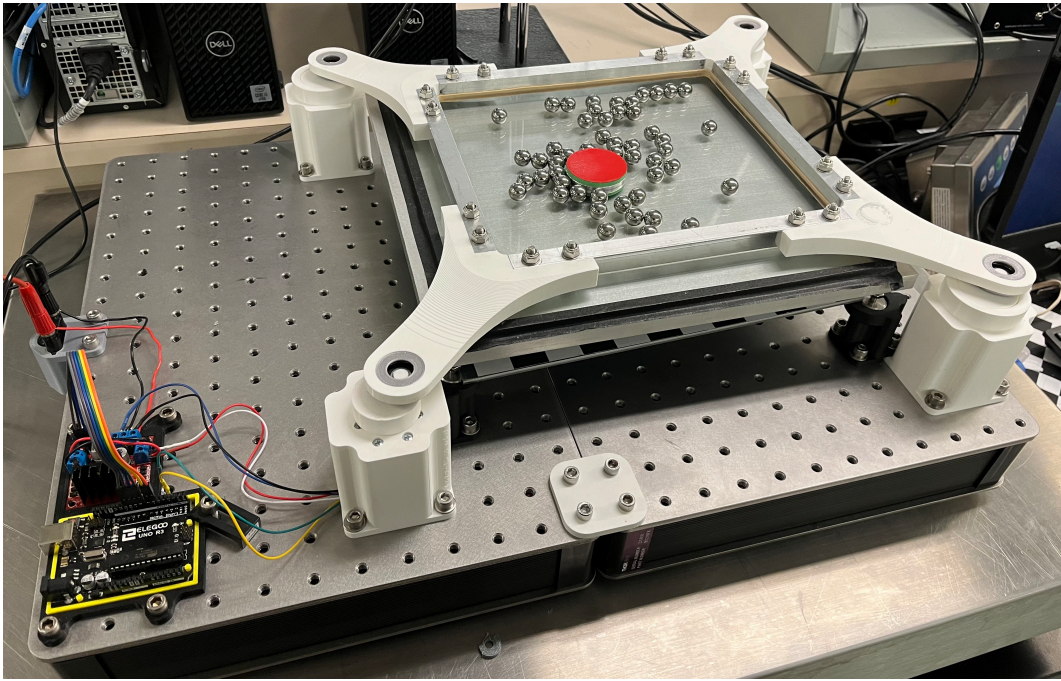


Figure 11: Alternate view of the actuation frame, actuation feet, actuation corners, optical and levelling tables, puck and steel balls, glass surface and Arduino connection.

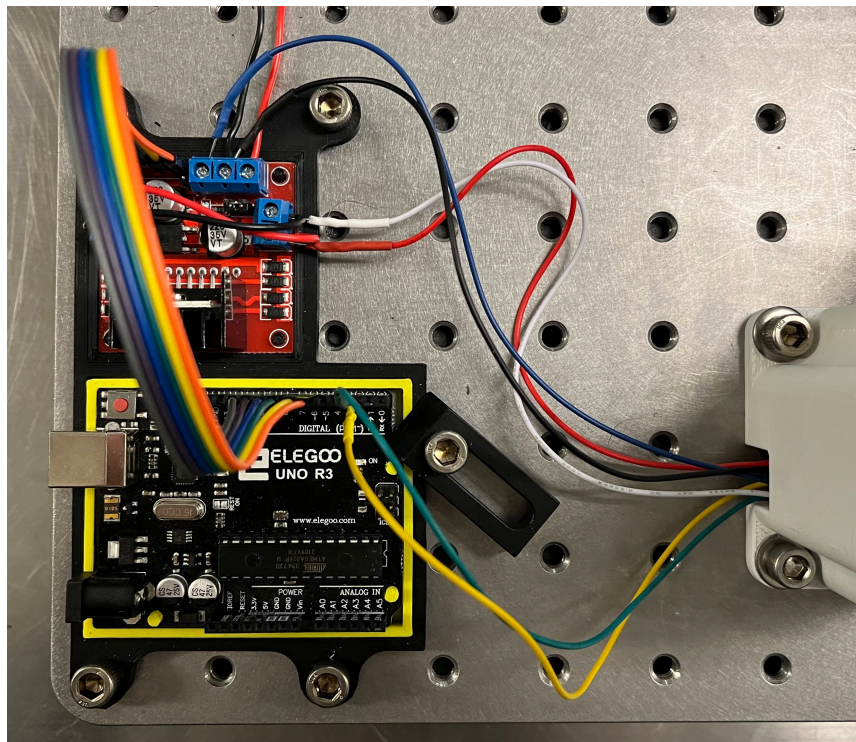


Figure 12: The connection of the Arduino controller to the motor that performs actuation.